

Mathematical Foundations of Data Lakes & Columnar Storage

Companion to Session 4 | Big Data Engineering Exercise Sheet (without solutions)

I/O Cost Models, Predicate Pushdown, Partition Pruning
and the Small-File Problem

Document Contents

- Cost model for column pruning: row vs. columnar storage.
- Predicate pushdown via row-group statistics; selectivity arithmetic.
- Partition pruning: directory-tree combinatorics.
- The small-file problem: metadata overhead vs. parallelism.
- Compression ratios and total cost of ownership.
- **Two exercises (statements only)** to be solved by the student.

Contents

1	Introduction	2
2	Column Pruning: A Cost Model	2
2.1	Row-oriented vs. columnar I/O	2
2.2	The pruning ratio	2
2.3	Compression amplification	3
3	Predicate Pushdown via Row-Group Statistics	3
3.1	Parquet row-group skipping	3
3.2	Probability that a row group is skipped	3
3.3	When does pushdown help most?	4
4	Partition Pruning: Directory Combinatorics	4
4.1	The hierarchical partition tree	4
4.2	Pruning under filter conditions	4
4.3	Compounding with column pruning and pushdown	5
5	The Small-File Problem	5
5.1	Why file count matters	5
5.2	The crossover: when overhead dominates	6
5.3	The small-file budget formula	6
6	Storage & Compression Cost Model	6
6.1	Total cost of ownership (storage)	6
7	Exercises	8
8	Summary	10
8.1	Going further	10

1. Introduction

Session 4 sits at the boundary between data engineering and systems performance: every choice you make about storage format, compression, and partitioning translates directly into bytes read from disk, requests sent to S3, and dollars on the cloud bill. The right way to reason about those choices is *quantitatively*.

We focus on five mathematical building blocks:

1. **Column pruning** as a fraction-of-bytes calculation $\rightarrow$ how much I/O is saved by reading k of n columns.
2. **Predicate pushdown** as a selectivity model $\rightarrow$ probability that a row group survives a filter.
3. **Partition pruning** as a tree-traversal cost $\rightarrow$ directories visited as a function of query selectivity.
4. **Small-file overhead** $\rightarrow$ when does file count, not data volume, dominate query cost?
5. **Compression & storage TCO** $\rightarrow$ ratio comparisons between Parquet/ORC/CSV and their dollar implications.

2. Column Pruning: A Cost Model

2.1 Row-oriented vs. columnar I/O

Consider a dataset with N rows and n columns. Let w_j denote the average byte width of column j , and let

$$W = \sum_{j=1}^n w_j$$

be the total row width. The dataset's total size is $N \cdot W$.

Definition – Bytes Read for a Query

A query reading columns $S \subseteq \{1, \dots, n\}$ over the full dataset reads:

- **Row format** (CSV, JSON, Avro): the entire dataset must be scanned because rows are interleaved.

$$B_{\text{row}} = N \cdot W.$$

- **Columnar format** (Parquet, ORC): only the requested columns are read.

$$B_{\text{col}}(S) = N \cdot \sum_{j \in S} w_j.$$

2.2 The pruning ratio

The fraction of bytes saved by column pruning is

$$\rho_{\text{prune}}(S) = 1 - \frac{\sum_{j \in S} w_j}{W}.$$

Reading $|S| = k$ columns of equal width yields a pruning ratio of $1 - k/n$.

Example. A wide event table has $n = 30$ columns of equal width $w_j = 8$ bytes (so $W = 240$ bytes/row). A query selects only 4 columns:

$$\rho_{\text{prune}} = 1 - \frac{4 \cdot 8}{240} = 1 - \frac{32}{240} \approx 86.7\%.$$

The columnar format reads $\sim 1/7$ of the bytes the row format would.

2.3 Compression amplification

Columnar formats compress better because each column has a single data type with high redundancy. Let r_{col} be the typical column compression ratio (e.g. $4\times$) and r_{row} the row compression ratio ($\sim 2\times$).

Effective Bytes Read

After compression, the bytes read are

$$B_{\text{row}}^{\text{comp}} = \frac{N \cdot W}{r_{\text{row}}}, \quad B_{\text{col}}^{\text{comp}}(S) = \frac{N \cdot \sum_{j \in S} w_j}{r_{\text{col}}}.$$

The total speed-up over uncompressed CSV is therefore

$$\text{Speed-up} = \frac{B_{\text{row}}}{B_{\text{col}}^{\text{comp}}(S)} = \frac{W}{\sum_{j \in S} w_j} \cdot r_{\text{col}}.$$

In the wide-table example with $r_{\text{col}} = 4$:

$$\text{Speed-up} = \frac{240}{32} \cdot 4 = 7.5 \cdot 4 = 30 \times .$$

3. Predicate Pushdown via Row-Group Statistics

3.1 Parquet row-group skipping

Parquet stores each column chunk's min/max/null_count in the file footer. When a query has a filter `WHERE col > v`, the engine *skips* any row group whose stored max < v .

Definition – Selectivity of a Predicate

The **selectivity** of a predicate ϕ is

$$\sigma(\phi) = \mathbb{P}(\text{a random row satisfies } \phi).$$

A query with $\sigma = 0.05$ keeps 5% of the rows.

3.2 Probability that a row group is skipped

Suppose a column has values drawn i.i.d. from a distribution F . A row group of G rows has minimum $\min_{i=1}^G X_i$ and maximum $\max_{i=1}^G X_i$. For a filter `col > v`, the row group is

skippable iff its $\max \leq v$, i.e. all G values fall below v :

$$\mathbb{P}(\text{skip}) = F(v)^G = (1 - \sigma)^G.$$

X¹ Theorem – Effective Bytes Read After Pushdown

With R row groups of G rows each, the expected fraction of row groups *read* is

$$\mathbb{E}[\text{fraction read}] = 1 - (1 - \sigma)^G.$$

Therefore, the expected bytes read for a single column with simple inequality filter is

$$\mathbb{E}[B_{\text{read}}] \approx N \cdot w_j \cdot (1 - (1 - \sigma)^G).$$

3.3 When does pushdown help most?

- **Tiny selectivity** ($\sigma \rightarrow 0$): $(1 - \sigma)^G \rightarrow 1$, so almost every row group is skipped. Pushdown is highly effective.
- **Large selectivity** ($\sigma \rightarrow 1$): no row group can be skipped. Pushdown gains nothing.
- **Sorted data**: matching values are concentrated in a few row groups; the binomial model becomes a step function and the skip rate approaches $1 - \sigma$ rather than $1 - (1 - \sigma)^G$.

i Important Remark

The bound $1 - (1 - \sigma)^G$ assumes *uniform* placement of matching rows across row groups. **Sorting the data on the filter column before writing** dramatically improves pushdown – this is the principle behind “Z-ordering” in Delta Lake and “cluster-by” in Iceberg.

4. Partition Pruning: Directory Combinatorics

4.1 The hierarchical partition tree

A data lake partitioned on d columns with $c_1, c_2, \dots, c_d$ distinct values per level forms a tree with

$$D = \prod_{i=1}^d c_i$$

leaf directories. Without partition pruning, all D directories must be listed and read.

4.2 Pruning under filter conditions

A query with predicates on partition columns reduces the tree to a sub-tree. Let $f_i \in [0, 1]$ be the fraction of values in level i that the query keeps (e.g. $f_i = 1/12$ to filter on a single month). If filters on different levels are independent:

X¹ Theorem – Partition Pruning Ratio

The fraction of directories that must be read is

$$\rho_{\text{read}} = \prod_{i=1}^d f_i.$$

The amount of data read is

$$B_{\text{partition}} = \rho_{\text{read}} \cdot N \cdot W$$

(times any column-pruning factor on top).

Example. A lake partitioned by `sensor_type` / `year` / `month` / `day` with respectively 5, 3, 12, 30 values has

$$D = 5 \cdot 3 \cdot 12 \cdot 30 = 5400 \text{ leaf directories.}$$

A query for *temperature in January 2024* over 7 days keeps

$$f_1 = 1/5, \quad f_2 = 1/3, \quad f_3 = 1/12, \quad f_4 = 7/30.$$

$$\rho_{\text{read}} = \frac{1}{5} \cdot \frac{1}{3} \cdot \frac{1}{12} \cdot \frac{7}{30} = \frac{7}{5400} \approx 0.13\%.$$

The query touches only 0.13% of the data – a $\sim 770\times$ speed-up over a non-partitioned scan.

4.3 Compounding with column pruning and pushdown

When all three optimisations apply, the bytes read multiply:

$$B_{\text{final}} = \rho_{\text{read}} \cdot \frac{\sum_{j \in S} w_j}{W} \cdot (1 - (1 - \sigma)^G) \cdot N \cdot W \cdot \frac{1}{r_{\text{col}}}.$$

This compound formula is the central scan-cost model of every modern analytical engine (Spark, Trino, BigQuery, Snowflake).

5. The Small-File Problem

5.1 Why file count matters

Reading a file from a distributed file system has two cost components:

- A **fixed overhead** τ per file (open, metadata fetch, list operation): typically a few milliseconds locally, 50–200 ms on S3.
- A **streaming throughput** β for the bytes inside.

Definition – Total Read Time for F Files

For F files of total size B bytes:

$$T_{\text{read}}(F, B) = F \cdot \tau + \frac{B}{\beta}.$$

5.2 The crossover: when overhead dominates

The fixed term $F\tau$ dominates the streaming term B/β when $F\tau > B/\beta$, i.e. the average file size B/F is smaller than $\tau\beta$.

Optimal Average File Size

Let $s_{\text{opt}} = \tau\beta$. Files smaller than s_{opt} are dominated by metadata overhead; larger files are dominated by streaming.

Example. On AWS S3 with $\tau = 100$ ms and $\beta = 100$ MB/s:

$$s_{\text{opt}} = 0.1 \text{ s} \cdot 100 \text{ MB/s} = 10 \text{ MB}.$$

That's a *lower* bound: files of 10 MB are equally dominated by both costs. The Parquet community recommendation of 128 MB to 1 GB ensures the streaming term clearly dominates, so per-file overhead is amortised.

5.3 The small-file budget formula

Fix a target dataset size B and average file size s :

$$F = B/s.$$

The total read time becomes

$$T_{\text{read}}(s) = \frac{B}{s}\tau + \frac{B}{\beta} = B \cdot \left(\frac{\tau}{s} + \frac{1}{\beta} \right).$$

At $s = s_{\text{opt}}$, the two terms are equal. Halving the file size *doubles* the metadata cost. This gives the small-file problem its characteristic shape: a 1 TB dataset stored in 10 million files takes 10× longer to scan than the same TB in 100 000 files.

6. Storage & Compression Cost Model

6.1 Total cost of ownership (storage)

Let:

- V = uncompressed dataset size (bytes);
- r = compression ratio (so the file on disk is V/r bytes);
- p = price per GB-month of storage.

The monthly storage cost is

$$\text{Cost}_{\text{storage}} = \frac{V}{r} \cdot p.$$

For a 10 TB dataset on S3 at \$0.023/GB-month:

- CSV ($r = 2$): 5,000 GB $\cdot$ \$0.023 = \$115/month.
- Parquet+Snappy ($r = 6$): $\sim$ \$38/month.
- Parquet+Zstd ($r = 10$): $\sim$ \$23/month.

Compression savings are linear; over a year the difference is hundreds of dollars per TB of historical data.

7. Exercises

The two exercises that follow combine the building blocks above. Every numerical value needed is given in the statement.

Exercise 1 – Column Pruning, Predicate Pushdown, and Compound Speed-up

A data engineering team stores a 1 TB clickstream dataset in two formats:

- **CSV** (row-oriented, gzip): compression ratio $r_{\text{row}} = 3$.
- **Parquet+Snappy** (columnar): compression ratio $r_{\text{col}} = 6$.

The schema has $n = 25$ columns of average width $w = 16$ bytes per row, so the row width is $W = 400$ bytes. The dataset contains $N = 2.5 \times 10^9$ rows. The Parquet file uses row groups of $G = 100,000$ rows.

Part A. Column pruning. A typical analytics query reads only $|S| = 5$ columns out of 25 to compute a session-level aggregate.

- Compute the bytes read by a CSV scan (compressed).
- Compute the bytes read by a Parquet scan with column pruning but *no* predicate pushdown.
- Compute the column-pruning ratio ρ_{prune} .
- Compute the speed-up of Parquet over CSV from column pruning and compression alone.

Part B. Predicate pushdown. The same query also has a filter `WHERE country_code = 'FR'`. Out of all rows, $\sigma = 1/50$ have `country_code = 'FR'`, and country codes are randomly distributed across row groups (no sorting).

- Compute the probability that a single row group is *skipped* thanks to its min/max statistics. Comment.
- How many row groups are there in total?
- Now suppose the data is sorted by `country_code` before writing. What fraction of row groups must be read in this case? (Hint: contiguous matching rows.)
- Compute the bytes read by Parquet with column pruning + sorted pushdown. Compare to part (b).

Part C. Total speed-up.

- Compute the total speed-up of (sorted) Parquet over CSV for this query. Express as a multiplicative factor.

 Exercise 2 – Partition Design and the Small-File Problem

A retail company stores transaction logs in S3 as a Parquet data lake. The data is partitioned by

`country / year / month / day`

with the following cardinalities:

$$c_1 = 12 \text{ countries}, \quad c_2 = 5 \text{ years}, \quad c_3 = 12 \text{ months}, \quad c_4 = 31 \text{ days/month}.$$

Each day-leaf directory holds an average of 2 GB of compressed data, currently split into 200 files of average size 10 MB each.

Part A. Partition design.

- (a) Compute the total number D of leaf directories.
- (b) Compute the total dataset size V (in TB).
- (c) A typical query reads *transactions for France in 2024, between March 1 and March 7*. Compute the partition-pruning ratio ρ_{read} .
- (d) How much data is read at the partition level?

Part B. Small-file problem. S3 has fixed per-file overhead $\tau = 100$ ms and streaming throughput $\beta = 200$ MB/s per request thread.

- (e) Compute the optimal average file size s_{opt} .
- (f) For the query in (c), how many files F must be read with the *current* layout of 200 files per day-partition?
- (g) Compute the total read time T_{read} for the query under the current layout.
- (h) The team runs a compaction job that rewrites each day-partition as a single 2 GB file. Compute the new total read time. What speed-up is achieved?

Part C. The high-cardinality trap. The company is asked to add `user_id` (cardinality 10 million) as a partition column, between `country` and `year`.

- (i) Compute the new total number of leaf directories.
- (j) If the dataset size remains V as in (b), what is the average file size in each leaf directory, assuming the same number of files per leaf? Comment.
- (k) Briefly justify mathematically why the proposal must be **rejected**.

8. Summary

Key Formulas to Remember

Concept	Formula
Bytes read, row format	$B_{\text{row}} = N \cdot W$
Bytes read, columnar format	$B_{\text{col}}(S) = N \cdot \sum_{j \in S} w_j$
Column-pruning ratio	$\rho_{\text{prune}} = 1 - \frac{\sum_{j \in S} w_j}{W}$
Compression speed-up	$\frac{W}{\sum_{j \in S} w_j} \cdot \frac{r_{\text{col}}}{r_{\text{row}}}$
Row-group skip prob. (random)	$\mathbb{P}(\text{skip}) = (1 - \sigma)^G$
Row-group skip (sorted data)	fraction read $\approx \sigma$
Partition pruning ratio	$\rho_{\text{read}} = \prod_i f_i$
Total read time	$T_{\text{read}} = F\tau + B/\beta$
Optimal file size	$s_{\text{opt}} = \tau \cdot \beta$
Storage cost	$\text{Cost} = (V/r) \cdot p$

8.1 Going further

- Melnik, S. et al. *Dremel: Interactive Analysis of Web-Scale Datasets*, VLDB 2010 – the foundational paper for Parquet’s nested encoding.
- Apache Parquet documentation: parquet.apache.org/docs.
- Armbrust, M. et al. *Delta Lake: High-Performance ACID Table Storage*, VLDB 2020 – Z-ordering and small-file compaction.
- Kleppmann, M. *Designing Data-Intensive Applications*, Ch. 3 (Storage and Retrieval).